

מבוא לבניה מלאכותית - תרגול 8.

רגב יחזקאל אימרה.

12 בינואר 2026

1 אימון רשתות (עמוקות).

בבואנו לאמן רשת נוירונים, ישנם מספר בעיות שחוזרות על עצמן שוב ושוב, ולכן חשוב להכירן.

1.1 *Vanishing/Exploding gradients*

נניח ועכשיו אני מאמן רשת נוירונים עם 50 שכבות, ואני רוצה לעשות צעד *back-propagation*. אם הרשת שלי נראית משהו כמו

$$h_{n+1} = f(W_{n+1}h_n + b_{n+1})$$

אז אני אצטרך להכפיל בנגזרת שלי הרבה מאוד $\frac{\partial h_i}{\partial h_{i-1}}$, והבעיה שלי היא שברגע שאני מכפיל הרבה מאוד מספרים, המכפלה יכולה לעשות 2 דברים: 1. לקרוס ל-0: אם המוכפלים קטנים מ-1.

2. להתפוצץ ל- ∞ : אם המוכפלים גדולים מ-1.

כמובן ששתי התוצאות האלה לא טובות לנו בזמן האימון, כי אם תהיה לי נגזרת מאוד מאוד קטנה אז כשאני אעדכן את המשקולות שלי אני אראה

$$W' = W - \underbrace{\eta \frac{\partial Loss}{\partial W}}_{\approx 0} = W$$

כלומר לא עדכנתי את W , גם אם אני עדיין רחוק מלהגיע למינימום של $Loss$. אם תהיה לי נגזרת מאוד גדולה אזי אני אעדכן את W להיות משהו מאוד גדול ואז כל הרשת שלי תתפוצץ ותפלוט סתם ערכים גדולים. הפתרון יהיה לאתחל את המשקולות שלי בצורה חכמה!

1.1.1 *Weight Initialization*

הרעיון הוא שאנחנו רוצים שהשונות של הקלט יהיה שווה לשונות של הפלט כדי שהשכבה לא תתפוצץ/תדעך. אם יש לי $y = Wx$ אזי

$$\mathbb{V}[y_i] = \mathbb{V}\left[\sum_{j=1}^{d_{in}} x_j w_j\right] \stackrel{*}{=} d_{in} \mathbb{V}[x_i w_i] \stackrel{**}{=} d_{in} \left(\mathbb{E}[x_i^2] \mathbb{E}[w_i^2] - \mathbb{E}[x_i]^2 \mathbb{E}[w_i]^2 \right) \stackrel{***}{=} d_{in} \mathbb{V}[x_i] \mathbb{V}[w_i]$$

לכן אם ניקח $\mathbb{V}[w_i] = \frac{1}{d_{in}}$ נקבל $\mathbb{V}[y_i] = \mathbb{V}[x_i]$.

* נניח x, w בלתי תלויים שווי התפלגות.

** $x \perp w$ נניח x, w בעלי תוחלת 0.

*** נניח x, w בעלי תוחלת 0.

נעיר כי זו לא שיטה חסרת תקלות: הרי אנחנו מניחים ש- w ממורכז סביב 0, אבל אחרי שכבת $ReLU$ אחת, כל מה שמתחת ל-0 יהפוך ל-0 מה שאומר שלא יהיו תוצאות שליליות....

המחקר בתחום הזה עדיין מאוד פעיל וכל שנה צצות שיטות חדשות איך לאתחל את המשקולות בצורה נכונה.

עוד דרך בה אפשר להגיע לבעיה הזו היא בעזרת פונקציות אקטיבציה שנהיות רוויה: קחו לדוגמה את $\sigma(x) = \frac{1}{1+e^{-x}}$. מה קורה עבור $x = \pm 4$? כבר אז אנחנו מקבלים תוצאה שקרובה ל-1, לכן הנגזרת, שהיא $\sigma(x)(1 - \sigma(x))$ תהיה קרובה ל-0. הפתרון: להשתמש בפונקציית אקטיבציה שלא נהיית רוויה, כמו $ReLU$ ושאר הפונקציות מאותה משפחה.

1.1.2 *Batch Normalization*

דרך שלישית לגרום לנגזרות לא להתפוצץ היא בעזרת *Batch Normalization*: ניקח *batch* מהדגימות שלנו בשלב מסוים ונחשב להם

$$\mu_B = \frac{1}{m_B} \sum_{i=1}^{m_B} x^i$$
$$\sigma_B^2 = \frac{1}{m_B} \sum_{i=1}^{m_B} (x^i - \mu_B)^2$$

$$\hat{x}^i = \frac{x^i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

על מנת לתת לרשת ללמוד את ההתפלגות האופטימלית של הדאטה, נציג 2 פרמטרים חדשים לתהליך הלמידה: β, γ ואז נחשב

$$z^i = \gamma \hat{x}^i + \beta$$

וניתן לרשת ללמוד גם אותם.

את כל התהליך הזה עושים מיד לפני/ מיד אחרי פונקציית האקטיבציה של השכבה החבויה, ועושים אותו בכל השכבות.

1.1.3 Gradient Clipping

הפתרון הרביעי שאנחנו יכולים לנסות הוא *Gradient Clipping*: להכריח את הנגזרת שלנו להיות עד טווח מסויים. נניח אני לא רוצה נגזרת שבערך מוחלט תהיה מעל 5, אז אני אקבע שכל נגזרת שלי תהיה

$$dw = \begin{cases} -5 & dw < -5 \\ dw & -5 \leq dw \leq 5 \\ 5 & 5 < dw \end{cases}$$

וכך אני יכול באופן מלאכותי לשלוט על גודל הנגזרות שלי. ככה למשל, הגרדיאנט

$$(-6, 0.7, 5.2, 3.4, 8)$$

יהפוך ל-

$$(-5, 0.7, 5, 3.4, 5)$$

אבל נשים לב כי איבדנו את הכיוון של הגרדיאנט הזה, לכן אם נרצה לשמר את הכיוון הזה, לא נקבע מקסימום לערכים, אלא מקסימום לנורמה, וננרמל את הגרדיאנט כדי שנגיע למקסימום הזה $\nabla w = \nabla w \frac{thr}{\|\nabla w\|}$.

1.2 Regularization

עכשו נניח שהרשת מתאמנת, אבל יש *Overfit*! איך אנחנו יכולים למנוע מהרשת לשנן את הדאטה, וללמוד אותו במקום?

1.2.1 Dropout

האפשרות הראשונה היא באמצעות *dropout*: הרעיון הוא שבכל *forward*, נכבה ברנדומליות נוירונים ברשת (נקבע אותם להיות 0). ההסתבכות לכיבוי היא היפרפרמטר - לרוב נהוג 0.5. הרעיון כאן היה ששום נוירון לא צריך "להשתלט" על הרשת ולצבור יותר מדי כוח לבדו, ושלא יש נוירון כזה והוא יכבה, הרשת תצטרך ששאר הנוירונים ילמדו במקומו וככה לא יסתמכו עליו. חשוב שאותו פרמטר כיבוי יבכר לאורך כל השכבות, ולא שיהיה פרמטר אחר לכל שכבה.

1.2.2 Regularization

עוד שיטה שכבר מוכרת לנו למניעת *overfit* היא הוספת קנס לפונקציית ה-*Loss*, כלומר

$$Total Loss = NN Loss + \lambda R(W)$$

כאשר פעמים רבות משתמשים ב-

$$L2 regularization \text{ ואז } R(W) = \sum_k \sum_l W_{k,l}^2 \text{ (נקרא גם Weight decay).}$$

$$L1 regularization \text{ ואז } R(W) = \sum_k \sum_l |W_{k,l}|$$

$$Elastic net(L1 + L2) \text{ ואז } R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$$

2 Optimizers

אנחנו יודעים שכדי לאמן מודל משתמשים ב-*GD*, אבל על מנת לאמן רשת נוירונים צריך כמות גדולה של דאטה, מה שאומר שלפעמים לעשות *GD* יקח הרבה מאוד זמן. לכן אנחנו מחפשים שיטות אחרות לאמן את המודל שלנו, כל שיטה כזו נקראת *optimizer*.

דוגמה 1.2. *SGD* הוא *optimizer* אחר, שעובד יותר מהר מ-*GD* במחיר שפה אנחנו לא עושים גרדיאנט על כל הדאטה, אלא על חלק ממנו. כדי לפתור חלק מהבעיות של *SGD* הציעו להכניס מומנטום לעדכון:

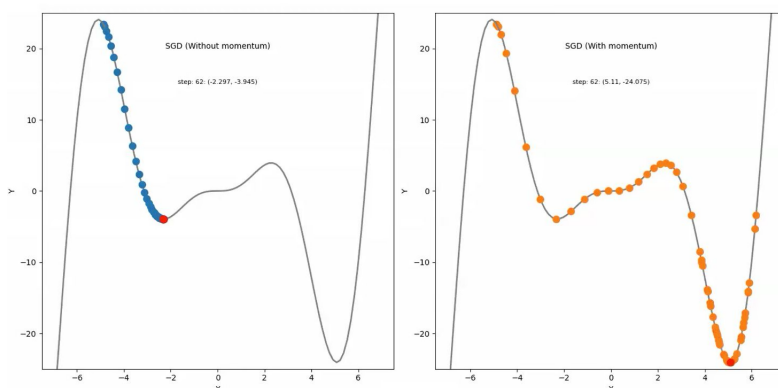
2.1 Momentum

הרעיון הוא שאם יש לנו ציר אחד שהוא מאוד איטי וציר אחד שהוא מאוד מהיר, אז היינו רוצים שבציר האיטי כל עוד אני נע באותו כיוון שאני אצבור שמה מהירות ואנוע יותר מהר. לעומת זאת אם אני כל איטרציה נע בכיוון אחר אז אני לא צריך לקבל מומנטום. נוסחאת העדכון נראית ככה:

$$x_{t+1} = x_t - \alpha v_{t+1}$$

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

הרעיון פה זה שככל שאני יותר איטרציות עם $\nabla f(x_t)$ בכיוון של v_t , ככה v_{t+1} ימשיך לעלות ואז ההתקדמויות יהיו מהירות יותר ויותר באותו ציר. מומנטום אפילו יכול לעזור לברוח ממינימום מקומי!



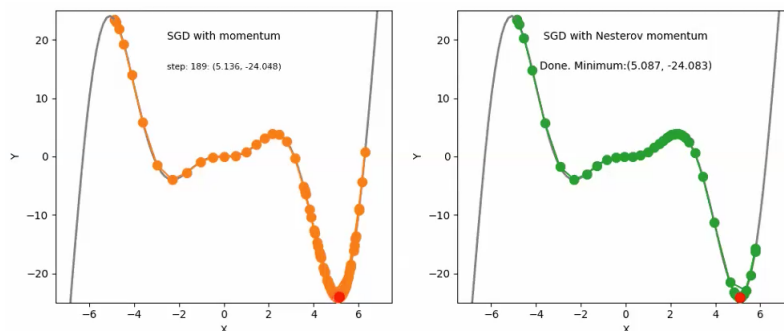
בעיות: יכול להיות ש- v יצבור יותר מדי מהירות ואז יעבור את המינימום המקומי ובאיטרציות הבאות נצטרך לחזור לכיוון הנגדי כדי להגיע למינימום.

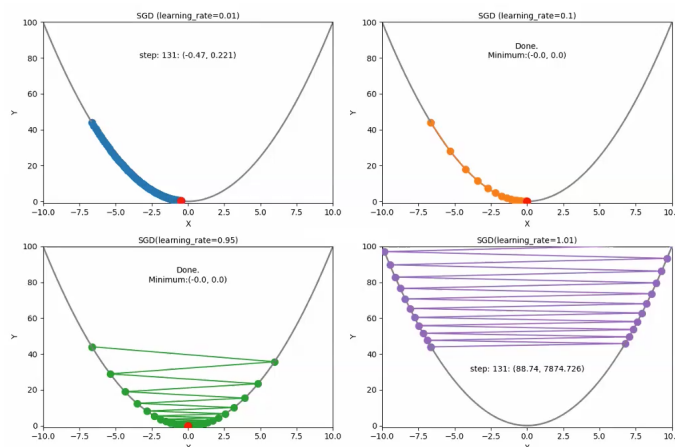
2.2 Nesterov Momentum

במקום לחשב את הגרדיאנט ב- x_t שלי, אני מחשב את הגרדיאנט איפה שאני אמור להיות $(x_t + \rho v_t)$, וככה אני "מסתכל קדימה" על הנקודה שמומנטום יקח אותי אליה כדי להבין אם אני הולך לפספס מינימום לוקאלי. בנוסחאות זה

$$x_{t+1} = x_t + v_{t+1}$$

$$v_{t+1} = \rho v_t - \alpha \nabla f(x_t + \rho v_t)$$





בתמונה רואים השוואה בין 4 קצבי למידה: 2 גדולים ו-2 קטנים. אם lr גבוהה אז העדכוניים שלנו יקפצו ממקום למקום. אם נקבע את lr להיות 0.1 אז נגיע למינימום לאט, ואם נקבע אותו להיות 0.01 נגיע אליו ממש ממש לאט. איך אני יודע אם קבעתי lr גבוה מדי/ נמוך מדי? באותו אופן בדיוק, אם המודל שלי מתאמן ואני רואה את ה- $loss$ מקפץ אז ה- lr גבוה מדי ואם ה- $loss$ לא זז אז ה- lr נמוך מדי. עכשיו מה קורה אם יש לי ציר שבו ה- lr גבוה מדי ובציר אחר ה- lr נמוך מדי? בשביל זה המציאו את AdaGrad שבה להתאים lr שונה לכל ציר. בנוסחאות זה

$$v_t = v_{t-1} + (\nabla f(w_t))^2$$

$$x_{t+1} = x_t - \frac{\alpha}{\sqrt{v_t + \epsilon}} \nabla f(w_t)$$

החילוק של α ב- $\sqrt{v_t + \epsilon}$ אומר שבכיוונים שאני ממש מהיר בהם ה- lr הכולל יקטן (כי v_t יהיה גדול) ובכיוונים שאני איטי בהם ה- lr הכולל יגדל (כי v_t יהיה קטן). הבעיה: v_t תמיד עולה ועולה, כלומר ה- lr תמיד קטן, ויש מצב שניעצר ונגיע ל- $lr = 0$ ועוד לא הגענו למינימום.

2.4 RMSProp

נועד למנוע את הבעיה של AdaGrad שמשמשים לחלק את α בערך שרק עולה. עושה את זה בדרך הבאה:

$$x_{t+1} = x_t - \frac{\alpha}{\sqrt{v_t + \epsilon}} \nabla f(w_t)$$

$$v_t = \beta v_{t-1} + (1 - \beta)(\nabla f(w_t))^2$$

β נקרא *decay rate*

הרעיון פה הוא שכל עוד $(\nabla f(w_t))^2$ גדול, v_t ילך וימשיך לגדול, אבל אם אנחנו לא זזים ממש מהר אז הגורם הדומיננטי הוא βv_{t-1} כלומר v_t יקטן בחזרה.

2.5 Adam

זה שילוב של RMSProp ו-Momentum, גם להאיץ את הלמידה בצירים שצריך אבל גם לבטל את הרעש שיש בזה ולא לעשות יותר מדי *overshot*. הנוסחאות יותר מסובכות:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla f(w_t) \quad \text{Momentum}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla f(w_t))^2 \quad \text{RMSProp}$$

היינו רוצים לפשט את הנוסחא של Adam ולכתוב

$$x_{t+1} = x_t - \frac{\alpha}{\sqrt{\hat{v}_t + \epsilon}} m_t$$

אבל יש עניין טכני קטן שאומר ש- v_t, m_t נוטים להיות מוטים לעבר 0 בצעדים הראשונים, לכן צריך לתקן אותם לפי

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

(β_i^t אומר β_i בחזקת t) ואז אפשר לכתוב

$$x_{t+1} = x_t - \frac{\alpha}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$$